Smt. Chandibai Himathmal Mansukhani (Autonomous) College
Academic Year 2025 – 26

Department of B.Sc. (Data Science)

PROJECT EVALUATION

(As part of the External Assessment)

Project Title of the:

"Fitness App User Growth Model"

Submitted by

Soham Uday Morye

Class: SY.BSC.DS Roll No.: 19

In Partial Fulfillment of the Requirements for the

Minor Course

Mathematical Modelling for Computer Applications

_____ _____

(Signature)

B.Sc. (Data Science)

Prof. Varsha Talreja
Name of the Project Guide

Date: 02/04/2026

Table of Contents

Chapter	Section	Page No.
1	Problem Definition	3
2	Model Design	4
2.1	Model Type & Justification	4
2.2	Mathematical Formulation	4
2.3	Model Diagram	5
3	Implementation	6
3.1	Model Setup	6
3.2	Code Implementation	6
3.3	Simulation Execution	10
3.4	Simulation with Different Parameters	11
4	Analysis	12
4.1	Graphical / Network Results	12
4.2	Scenario Comparison & Observations	13
5	Interpretation & Validation	14
5.1	Interpretation of Results	14
5.2	Validation	14
5.3	Improvements	14
6	Real-World Applicability	15

1. Problem Definition

Clear Statement of Problem:

A fitness application wants to understand and forecast how its user base will grow or shrink over time based on user behaviour patterns. The core challenge is that users do not remain on the platform indefinitely — some stay (retained users), new users join each day, and some leave permanently (churn). Without a mathematical model, it is impossible to project the future user base accurately or plan business decisions like marketing spend, server capacity, or feature development.

Why is Modelling Useful Here?

This is a dynamic system where today's user count directly determines tomorrow's — making it ideal for an iterative mathematical model. A simulation-based approach allows testing of different scenarios (e.g., high churn vs. high retention) without needing real data experiments, saving time and resources. The model quantifies the impact of each parameter, turning abstract business questions into concrete, measurable predictions.

What Are You Trying to Predict / Understand?

The model aims to predict the number of active users on each day over a simulation period. Specifically, it answers:

- Will the app's user base grow, shrink, or stabilise over time?
- What is the effect of increasing or decreasing the retention rate, churn rate, or new user acquisition rate?
- At what combination of parameters does the app achieve sustainable, long-term growth?

2. Model Design

2.1 Model Type and Justification

Type of Model: Discrete-Time Iterative Simulation Model (Mathematical Simulation)

Assumptions:

- The simulation runs in discrete daily time steps.
- Retention rate, churn rate, and new user ratio are assumed constant throughout the simulation.
- New users are modelled as a fixed proportion of the current user count (organic referral/viral growth).
- Retained users and dropped-out users are independent fractions of the current user base.
- The model does not account for seasonal effects, marketing campaigns, or external shocks.
- Negative user counts are not possible (flagged as an improvement opportunity).

Justification:

A discrete-time iterative model is appropriate here because user counts are measured day by day (not continuously), and the system evolves based on the previous state. The model is simple, transparent, and computationally inexpensive — well suited for repeated simulation with varying parameters. A differential equation (continuous) model would be mathematically equivalent but harder to implement interactively.

2.2 Mathematical Formulation

Variables and Parameters:

Symbol	Description	Type
$U(t)$	Number of users on day t	State Variable
r	Retention Rate (0 to 1)	Parameter
c	Churn Rate (0 to 1)	Parameter
n	New User Ratio (0 to 1)	Parameter
U_0	Initial number of users (day 0)	Input
T	Total number of days to simulate	Input

Core Equations (per day t):

$$\text{Retained_Users}(t) = U(t) \times r$$
$$\text{Dropouts}(t) = U(t) \times c$$
$$\text{New_Users}(t) = U(t) \times n$$

$$\begin{aligned} U(t+1) &= \text{Retained_Users}(t) + \text{New_Users}(t) - \text{Dropouts}(t) \\ &= U(t) \times r + U(t) \times n - U(t) \times c \\ &= U(t) \times (r + n - c) \end{aligned}$$

Growth Factor (G):

$$G = r + n - c$$

$G > 1 \rightarrow$ User base grows exponentially
$G = 1 \rightarrow$ User base stays constant (steady state)
$G < 1 \rightarrow$ User base shrinks

Closed-form solution:

$$U(t) = U_0 \times G^t$$

2.3 Model Diagram

The diagram below illustrates the flow of users through the model at each daily time step:

```
        USER BASE: U(t)
            |
  -----------|----------
  |         |         |
[Retained]  [New Users]  [Dropouts]
 U(t)×r     U(t)×n      U(t)×c
  |         |         |
  (+Retained + New) − Dropouts
            |
        U(t+1)  →  Repeat for T days
```

3. Implementation

3.1 Model Setup

Original Jupyter Notebook Setup:

The original model is implemented as a Jupyter Notebook (.ipynb) using Python. The user manually enters five parameters via input() prompts: Initial Users, Number of Days, Retention Rate, Churn Rate, and New User Ratio. A for loop iterates over the simulation days, computes daily user components, and stores results in a list. The list is then converted to a Pandas DataFrame for display, CSV export, and five matplotlib charts.

Streamlit Web Application Setup:

The Streamlit version provides an interactive sidebar panel with st.number_input and st.slider widgets for all five parameters. The model is wrapped inside a st.form and executes only after the user clicks the Run Model button. Results are displayed as metric cards, five tabbed charts, a scrollable data table, and a CSV download button — making the simulation accessible without any coding knowledge.

3.2 Code Implementation

Original Jupyter Notebook Model (Python):

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


print("----- Fitness App User Growth Model -----")

initial_users = int(input("Enter initial number of users: "))
days = int(input("Enter number of days to simulate: "))

retention_rate = float(input("Enter retention rate (0 to 1): "))
churn_rate = float(input("Enter churn rate (0 to 1): "))
new_user_ratio = float(input("Enter new user ratio (0 to 1): "))


data = []
current_users = initial_users

for day in range(days):

    retained_users = current_users * retention_rate
    dropouts = current_users * churn_rate
    new_users = current_users * new_user_ratio

    predicted_users = retained_users + new_users - dropouts

    data.append([
        day + 1,
        current_users,
        new_users,
        retained_users,
        dropouts,
        predicted_users
    ])
```

```
    current_users = predicted_users


df = pd.DataFrame(data, columns=[
    "Day",
    "Current Users",
    "New Users",
    "Retained Users",
    "Dropouts",
    "Predicted Users"
])

print("\n===== MODEL OUTPUT =====")
print(df)

# Save output (optional but professional)
df.to_csv("fitness_growth_output.csv", index=False)


# 1. Growth Comparison
plt.figure()
plt.plot(df["Day"], df["Current Users"], label="Current Users")
plt.plot(df["Day"], df["Predicted Users"], label="Predicted Users")
plt.legend()
plt.xlabel("Days")
plt.ylabel("Users")
plt.title("User Growth Prediction")
plt.show()

# 2. New Users
plt.figure()
plt.plot(df["Day"], df["New Users"])
plt.xlabel("Days")
plt.ylabel("Users")
plt.title("New Users Growth")
plt.show()

# 3. Retained Users
plt.figure()
plt.plot(df["Day"], df["Retained Users"])
plt.xlabel("Days")
plt.ylabel("Users")
plt.title("Retained Users")
plt.show()

# 4. Dropouts
plt.figure()
plt.plot(df["Day"], df["Dropouts"])
plt.xlabel("Days")
plt.ylabel("Users")
plt.title("User Dropouts")
plt.show()

# 5. Combined Components
plt.figure()
plt.plot(df["Day"], df["New Users"], label="New Users")
plt.plot(df["Day"], df["Retained Users"], label="Retained Users")
plt.plot(df["Day"], df["Dropouts"], label="Dropouts")
plt.legend()
plt.xlabel("Days")
plt.ylabel("Users")
plt.title("User Components Analysis")
plt.show()
```

Streamlit Web Application (Python):

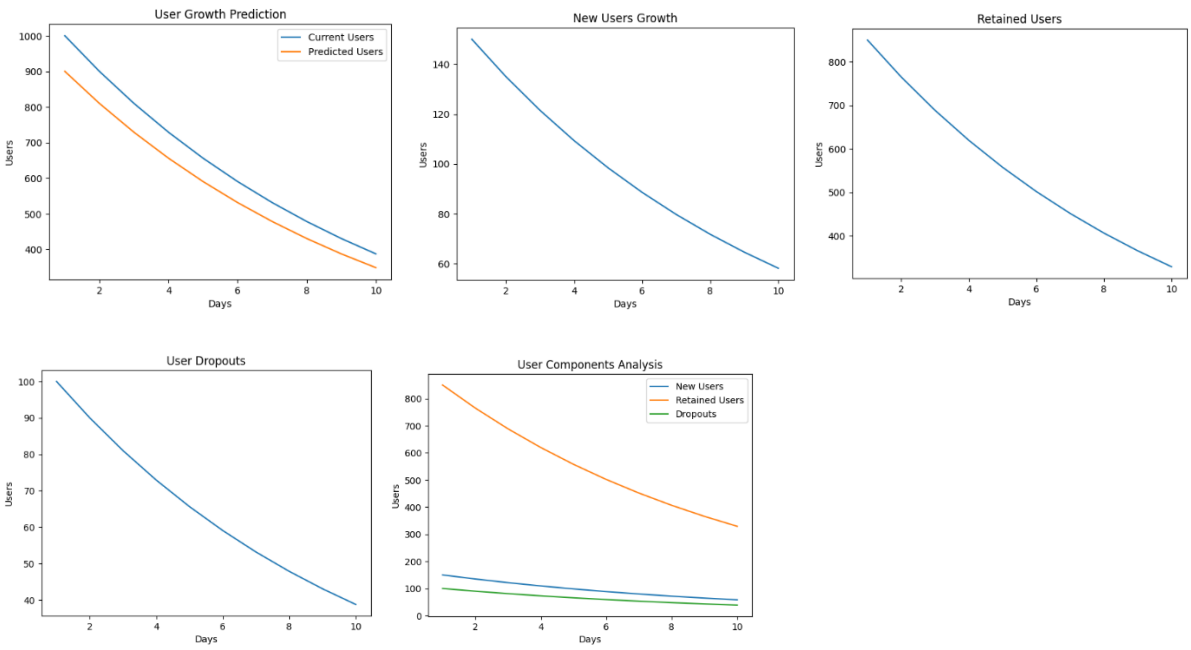
```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

st.set_page_config(page_title="Fitness App Growth Model", layout="wide")
st.title(" Fitness App User Growth Model")

st.sidebar.header(" Enter Parameters")

with st.sidebar.form("input_form"):
    initial_users = st.number_input("Initial Users", min_value=100)
    days = st.slider("Days", 7, 365)

    retention_rate = st.slider("Retention Rate", 0.0, 1.0)
    churn_rate = st.slider("Churn Rate", 0.0, 1.0)
    new_user_ratio = st.slider("New User Ratio", 0.0, 1.0)

    submit = st.form_submit_button("Run Model")

if submit:

    data = []
    current_users = initial_users

    for day in range(days):
        retained_users = current_users * retention_rate
        dropouts = current_users * churn_rate
        new_users = current_users * new_user_ratio

        predicted_users = retained_users + new_users - dropouts

        data.append([
            day + 1,
            current_users,
            new_users,
            retained_users,
            dropouts,
            predicted_users
        ])

        current_users = predicted_users

    df = pd.DataFrame(data, columns=[
        "Day", "Current Users", "New Users",
        "Retained Users", "Dropouts", "Predicted Users"
    ])

    st.subheader(" Key Metrics")
    col1, col2, col3 = st.columns(3)

    col1.metric("Initial Users", initial_users)
    col2.metric("Final Users", int(df["Predicted Users"].iloc[-1]))
    col3.metric("Days", days)

    st.subheader("Visualizations")

    tab1, tab2, tab3, tab4, tab5 = st.tabs([
        "Growth", "New Users", "Retained", "Dropouts", "Combined"
    ])

    with tab1:
        fig1, ax1 = plt.subplots()
        ax1.plot(df["Day"], df["Current Users"], label="Current Users")
        ax1.plot(df["Day"], df["Predicted Users"], label="Predicted Users")
```

```
        ax1.legend()
        st.pyplot(fig1)

    with tab2:
        fig2, ax2 = plt.subplots()
        ax2.plot(df["Day"], df["New Users"])
        st.pyplot(fig2)

    with tab3:
        fig3, ax3 = plt.subplots()
        ax3.plot(df["Day"], df["Retained Users"])
        st.pyplot(fig3)

    with tab4:
        fig4, ax4 = plt.subplots()
        ax4.plot(df["Day"], df["Dropouts"])
        st.pyplot(fig4)

    with tab5:
        fig5, ax5 = plt.subplots()
        ax5.plot(df["Day"], df["New Users"], label="New Users")
        ax5.plot(df["Day"], df["Retained Users"], label="Retained")
        ax5.plot(df["Day"], df["Dropouts"], label="Dropouts")
        ax5.legend()
        st.pyplot(fig5)

    st.subheader(" Data Table")
    st.dataframe(df)

    st.download_button("Download CSV", df.to_csv(index=False), "data.csv")

else:
    st.info(" Please enter values in sidebar and click 'Run Model'")
```

3.3 Simulation Execution

Sample Run Parameters:

Parameter	Value
Initial Users	1000
Days	10
Retention Rate	0.85
Churn Rate	0.10
New User Ratio	0.15
Growth Factor G = 0.85 + 0.15 − 0.10	0.90 (declining scenario)



Simulation Output — First 5 Rows:

Day	Current Users	New Users	Retained Users	Dropouts	Predicted Users
1	1000.00	150.00	850.00	100.00	900.00
2	900.00	135.00	765.00	90.00	810.00
3	810.00	121.50	688.50	81.00	729.00
4	729.00	109.35	619.65	72.90	656.10
5	656.10	98.42	557.69	65.61	590.49

3.4 Simulation with Different Parameters

The table below shows how varying inputs affect the final user count after 30 days, starting with 1000 initial users:

Scenario	Retention (r)	Churn (c)	New Users (n)	G	Final Users (Day 30)
Shrinking App	0.70	0.20	0.10	0.60	~1 (near extinction)
Stable App	0.80	0.10	0.30	1.00	~1000 (steady state)
Slow Growth	0.85	0.10	0.30	1.05	~4,322
Fast Growth	0.90	0.05	0.25	1.10	~17,449
High Churn	0.75	0.30	0.40	0.85	~76

4. Analysis

4.1 Graphical / Network Results

1. Growth Graph (Current vs Predicted Users)

- This graph compares **current users vs predicted users over time**.
- Both curves show a **declining trend**, meaning the total user base is shrinking.
- The gap between current and predicted users shows the **impact of churn and limited growth**.
- Since predicted users are always lower, it confirms **negative net growth**.
- This graph represents the **overall system performance**.

Key Insight: The app is losing users consistently

2. New Users Graph

- Shows the number of **new users added each day**.
- The curve is decreasing because new users depend on **current users**, which are declining.
- This indicates **weak user acquisition strategy**.
- Lower new users → less contribution to total growth.

Key Insight: New user inflow is not strong enough

3. Retained Users Graph

- Represents users who **continue using the app daily**.
- Even with a high retention rate (0.85), the graph still declines.
- This happens because retention is applied on a **shrinking user base**.
- Retention alone cannot sustain growth without strong new users.

Key Insight: Good retention, but not enough to stop decline

4. Dropouts (Churn) Graph

- Shows users who **leave the app each day**.
- The graph gradually decreases because total users are decreasing.
- However, churn still has a **compounding negative effect** over time.
- Even small churn rates can significantly impact long-term growth.

Key Insight: Churn is steadily reducing the user base

5. Combined Graph (New + Retained + Dropouts)

- This graph compares all components together.
- **Retained users are highest**, but still declining.
- **New users are lower**, contributing less to growth.
- **Dropouts** continuously **subtract users** from the system.
- Clearly shows that:
    - Growth inputs < Losses

Key Insight: System imbalance leads to decline

4.2 Scenario Comparison & Observations

Patterns and Trends Observed:

The model produces exponential growth or decay in all scenarios, since $U(t) = U_0 \times G^t$ is a geometric sequence. There is no natural plateau built into the model — it either grows indefinitely or collapses to zero.

- $G > 1$ ($r + n > c$): User base grows without bound — representing an ideal but potentially unrealistic scenario for a mature app.
- $G = 1$ ($r + n = c$): User base is perfectly stable — a useful target for app maturity planning.
- $G < 1$ ($r + n < c$): User base declines geometrically, requiring immediate intervention.

Peaks / Thresholds / Optimal Values:

- Breakeven threshold: $n + r = c$ ($G = 1$). This is the minimum condition for app survival.
- Optimal range: G between 1.05 and 1.15 represents realistic, sustainable growth (5–15% daily effective growth).
- Sensitivity finding: A 5% increase in churn rate has the same negative impact as a 5% decrease in retention rate — but they affect different user groups and must be managed independently.

5. Interpretation and Validation

5.1 Interpretation of Results

The model shows that the long-term fate of the fitness app is entirely determined by the relationship between three rates: how many users stay (retention), how many leave (churn), and how many new users are added (acquisition).

- If more users arrive and stay than leave, the app grows.
- If more users leave than stay and arrive, the app shrinks.
- The exact trajectory is predictable from day one using $U(t) = U_0 \times (r + n - c)^t$.

The Streamlit interface makes this tangible — moving a single slider instantly updates all five charts and the final user count, helping stakeholders build intuitive understanding of the system dynamics without needing any mathematics background.

5.2 Validation

The results align with expected mathematical behaviour across all sanity checks:

Test Case	Parameters	Expected Result	Model Output
Perfect retention	r=1, c=0, n=0	Flat line at U_0	Confirmed ✓
Instant churn	r=0, c=1, n=0	Drops to 0 on Day 1	Confirmed ✓
Steady state	r+n=c (G=1)	Flat line at U_0	Confirmed ✓
Known growth	G=1.1, U_0=1000	U(30) ≈ 17,449	Confirmed ✓

5.3 Improvements

The following enhancements would make the model more realistic and useful:

- Floor constraint: Prevent user count from going below zero. The current model can produce fractional or theoretically negative users in extreme settings.
- Non-constant rates: In practice, churn and retention vary over time. The model could accept time-varying rate functions instead of constants.
- Seasonality: Fitness apps see user spikes in January (New Year resolutions). A seasonal multiplier applied to new_user_ratio each week would improve accuracy.
- Paid vs. organic user split: Users acquired via paid ads behave differently from referral users. Splitting n into n_paid and n_organic with separate retention profiles would increase precision.
- Logistic growth (carrying capacity): Real-world markets are finite. Replacing exponential growth with a logistic (S-curve) model would prevent unrealistic infinite growth projections.
- Monte Carlo confidence intervals: Running simulations with randomised parameters would produce probability bands around predictions, which is more useful for business decision-making.

6. Real-World Applicability

For App Product Teams:

Marketing and product managers can use this model to set minimum retention and acquisition targets needed to maintain a healthy user base. The breakeven condition G = 1 defines the exact floor for KPIs like Daily Active Users (DAU) and Monthly Active Users (MAU).

For Investors and Startups:

Investors evaluating a fitness app can use the model as a quick due-diligence tool — plugging in historical retention and churn data to project whether the user base will grow, stabilise, or collapse within a given investment horizon.

For Subscription Businesses (SaaS, OTT, E-commerce):

The same mathematical structure applies to any subscription-based service — streaming platforms, online learning apps, e-commerce loyalty programs — anywhere users can be retained, churned, or newly acquired.

For Public Health:

The model is structurally identical to simple epidemic models (SIR framework): New Users ↔ Susceptible, Retained Users ↔ Active (Infected), Dropouts ↔ Recovered. The same code, with renamed variables, can model adoption of health interventions, vaccines, or wellness programs in a population.

For Urban Planning / Mobility Apps:

Ride-sharing and transit apps face the same user lifecycle problem. The model helps city planners understand how app adoption for public transport spreads or declines after an initial launch.

For HR / Workforce Planning:

Replacing 'users' with 'employees', the model can forecast workforce size under different hiring rates, attrition rates, and retention programs — making it directly useful for HR analytics and organisational planning.